SEMI DESIGN

SystemVerilog Project Report

Digital Clock

Prepared by: Ali Irfan

Date: 08/07/2026

Github Repository: https://github.com/Ali246801232/systemverilog-digital-clock

TABLE OF CONTENTS

LIST OF FIGURES

ABSTRACT

This project report presents the design, implementation, and verification of a 24-hour digital clock in SystemVerilog. The clock is built from a clock divider that generates a 1 Hz enable pulse, two modulo-60 counters, one modulo-24 counter, and a clock module that combines these components into a single module. The design is verified using a UVM (Universal Verification Methodology) testbench package that implements with three test scenarios that all pass: the active-low reset, the counting of seconds, and a full-day check for rollovers.

1 Introduction

1.1 Background and Context

Digital timekeeping is fundamental to nearly all modern electronic systems, from real-time clocks in personal computers to time-stamping units in industrial computers. The ability to accurately track time is a critical requirement in all of these applications. In digital logic, time is typically kept through a combination of counters and clock dividers that use the system clock as a way to count at the slower scale of seconds.

SystemVerilog, standardized as IEEE 1800, is a hardware description and verification language widely adopted by the industry for applications in chip-design, semiconductors, and digital logic (IEEE, 2017). The Universal Verification Methodology (UVM), standardized as IEEE 1800.2, builds on SystemVerilog by providing a reusable framework for building constrained verification environments with modular components (IEEE, 2017).

1.2 Problem Statement

Embedded systems like controllers and embedded SoCs require accurate timekeeping throughout the day. However, they often need to prioritize software resources for higher-priority system tasks than a 24-hour clock. This means a software clock can drift greatly over time due to other processes using the same software resources. This creates risks for timing violations that depend on this now unreliable clock.

Thus, a hardware clock is needed to provide accurate timekeeping independently of the software.

1.3 Objectives

- Design and implement modular digital clock in SystemVerilog with counters for seconds, minutes, and hours, and a 1-second enable pulse.
- Build a reusable and extensible UVM testbench with a driver, monitor, scoreboard, and agent components.
- Verify and validate the implementation through targeted testing using the UVM testbench.

2 Project Description

2.1 Theory/Background

Three theoretical concepts from circuit design are used in the design of this digital clock: a clock divider, cascading counters, and an asynchronous active-low reset.

A clock divider is a circuit that takes a high-frequency input clock and produces a lower-frequency enable signal by counting clock cycles. It uses a division factor N, then after every N cycles of the input block, it asserts an enable pulse for one cycle. This allows a faster system clock to drive logic that must operate at a much slower rate.

A counter, specifically a modulo-n counter, is a sequential circuit that counts from 0 to $n-1$, and then wraps back to 0. It increments by 1 on every clock cycle when the enable input is asserted, and then when it reaches $n-1$, it asserts a rollover flag which causes the counter to be set to 0. This rollover flag can also be used as the enable flag for another counter, to cascade them such that one counter only increments once another counter has rolled over.

An asynchronous active-low reset is a signal that forces all sequential elements to a known state upon being driven; the known state is usually zero. Being asynchronous means that it does not wait for the lock edge, which makes it useful for ensuring a deterministic startup. Being active-low convention means that it asserts a reset when the signal is 0 and de-asserts it when the signal is 1.

2.2 Functional Description

The digital clock keeps time by counting edges of a stable oscillator. The stable oscillator in question is the system clock. However, the system clock is too fast, typically 100 MHz, which means it must be converted to reduce its rate to 1 cycle per second (1 Hz) for the digital clock. This is done using a clock divider, which uses a division factor of 100,000,000 to produce a 1 Hz enable pulse.

The pulse from this clock divider must then be used by three cascading counters for each unit of the clock. Two modulo-60 counters are needed for the seconds and minutes output, and one modulo-24 counter is needed for the hours output. First, the enable pulse is passed to the seconds counter to count up to 60. Then the rollover flag of the seconds counter is passed to the minutes counter to count up to 60 as well. Lastly, the rollover flag of the minutes counter is passed to the hours counter to count up to 24.

The last component of this digital clock is an asynchronous active-low reset. This is an input signal passed to the module alongside the system clock, that allows the clock to be reset to 00:00:00 when it is asserted.

3 Project Implementation

3.1 RTL Design

The RTL design of the digital clock consists of four modules in a file `rtl/digital_clock.sv`:

- `clk_divider`: the clock divider, that converts the system clock to a 1 Hz enable pulse.
- `mod60_counter`: the modulo-60 counter for seconds and minutes.
- `mod24_counter`: the modulo-24 counter for hours.
- `digital_clock`: the top module that combines the above to construct the digital clock.

The top module passes the same `clk` and `rst_n` signals to each of the modules to synchronize them, while the `en` and `rollover` signals are chained through the cascading counters. The output of the module is based on the outputs of each counter. The `sec` output is the `count` output of the seconds counter. The `min` output is the `count` output of the minutes counter. The `hr` output is the `count` output of the hours counter.

The full SystemVerilog source code for this is in Appendix A1, but Figure 1 contains a block diagram representing this definition in a visual form, as it shows how each submodule flows into the next. It shows how each step of the inputs of the top module are passed to each submodule, and how the outputs of each counter are passed through into the inputs of the next counter. This cascades through to determine when the next counter is enabled based on the rollover of the previous counter.

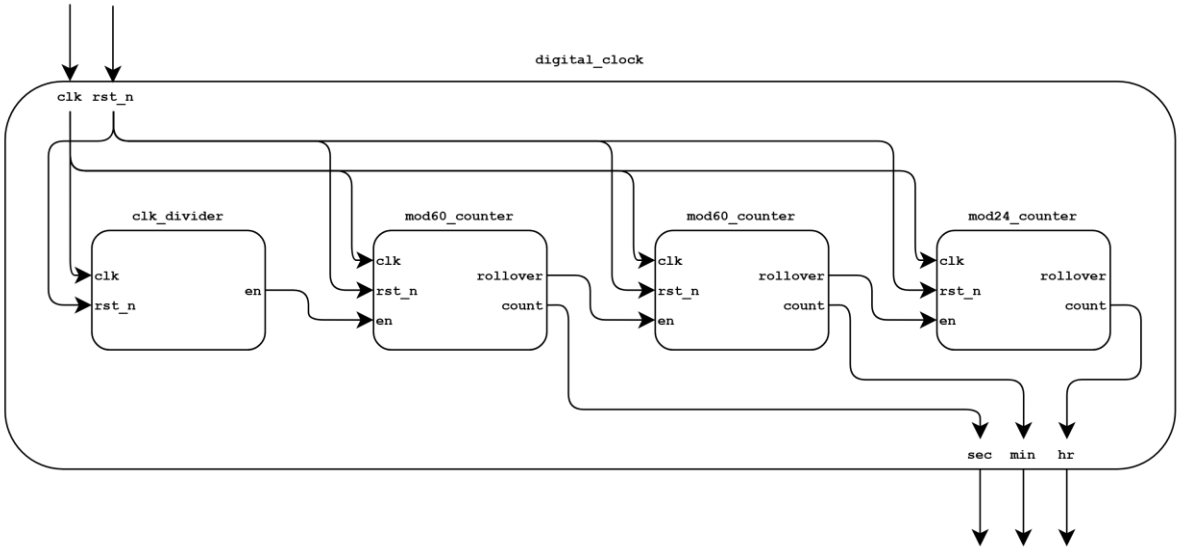

Figure 1: Block Diagram of `digital_clock`

3.2 Verification

The verification framework used to test the `digital_clock` module is defined across three components:

- `tb/digital_clock_if.sv`: The interface connecting the DUT and the testbench.
- `tb/clock_pkg.sv`: The complete UVM testbench package to test the DUT.
- `tb/top.sv`: The interface connecting the DUT and the testbench.

The full source code for all of these is available in Appendix A2, A3 and A4. The UVM testbench package contains all the necessary components, including the definition of a sequence item, a sequencer, a driver, a monitor, a scoreboard, an agent, and an environment.

While the majority of this is boilerplate, the driver and scoreboard have additional functions as well. The driver handles setting a `counting` parameter that controls when the pass and fail counts are updated, so that setup and cleanup phases of tests are not included in the checks. The scoreboard keeps track of a `pass_count` and `fail_count` that hooks back into the virtual interface, which allows the waveform to display them as well.

The tests themselves are defined as three test sequences, run by their respective test classes:

- `reset_seq -> reset_test`: 49 checks
  Drives `rst_n=0` for 15 clock cycles, then `rst_n=1` for 15 cycles. This verifies that the clock holds at 00:00:00 during reset and counts correctly after de-assertion.
- `seconds_seq -> seconds_test`: 59 checks
  After reset, drives `rst_n=1` for 59 clock cycles. This verifies that the second counter increments correctly once per cycle.
- `rollover_seq -> rollover_test`: 86400 checks
  After reset, drives `rst_n=1` for 86400 cycles (one full day). This exhaustively verifies:
  - Every seconds rollover from HH:MM:59 to HH:MM:00.
  - Every minutes rollover from HH:59:59 to HH:00:00.
  - The final hours rollover from 23:59:59 to 00:00:00.

These tests are all done with `DIV_FACTOR=1` to reduce the time of one clock cycle to 10 ns, which allows the testbench to test the digital clock without having to wait real-time seconds, which would greatly increase test time.

3.3 Simulation and Results

The simulation was performed on EDA Playground using the configuration shown in Figure 2. The version of the playground used is available at https://www.edaplayground.com/x/qgSM, and each test can be run by adding the `+UVM_TESTNAME` argument to the run options. Using this setup, all three tests were run one-by-one.

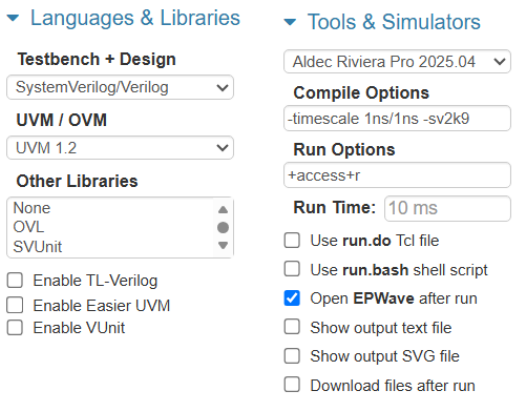

Figure 2: EDA Playground Configuration

The waveforms for each test generated by EDA Playground's simulator use EPWave to show the behaviour of 9 signals exposed by the top module:

- `top/clk`: The top module's system clock.
- `top/vif/sec[5:0]`: The virtual interface's seconds counter.
- `top/vif/min[5:0]`: The virtual interface's minutes counter.
- `top/vif/hr[4:0]`: The virtual interface's hours counter.
- `top/vif/rst_n`: The virtual interface's reset signal.
- `top/DIV_FACTOR[31:0]`: The top module's division factor (1 for fast tests).
- `top/vif/fail_count`: The number of tests failed.
- `top/vif/counting`: Whether test checking is enabled.

Figure 3 shows the waveform for `reset_test`. Here, the first 15 clock cycles assert `rst_n=0`, which we expect to cause the digital clock to stay at 00:00:00. Then, the next 15 clock cycles assert `rst_n=1`, which we expect to release the clock and allow it to count up to 15 seconds. The waveform shows exactly this.

Figure 4 shows the waveform for `seconds_test`. Here, the clock is first reset for 10 clock cycles to allow it to settle in a default state. Then, the test begins as reset is de-asserted and the clock is allowed to count up for 59 clock cycles. We expect it to count up to 59 seconds. The waveform shows exactly this.

Figure 5 shows three waveforms for `rollover_test`. Here, the clock is first reset for 10 clock cycles to allow it to settle in a default state. Then, the test begins as reset is de-asserted and the clock is allowed to count up for 86400 clock cycles, and we expect to count up for the full 24 hours. Along the way, we also expect it to rollover correctly at every 60 seconds, every 60 minutes, and at the final clock cycle at 24 hours. The first waveform shows the rollover from 00:00:59 to 00:01:00. The second waveform shows the rollover from 00:59:59 to 01:00:00. The third waveform shows the rollover from 23:59:59 to 00:00:00.

In total, this results in 30 passes for the first test, 59 passes for the second test, and 86400 passes for the third test. This is exactly what the pass count shows in each waveform.

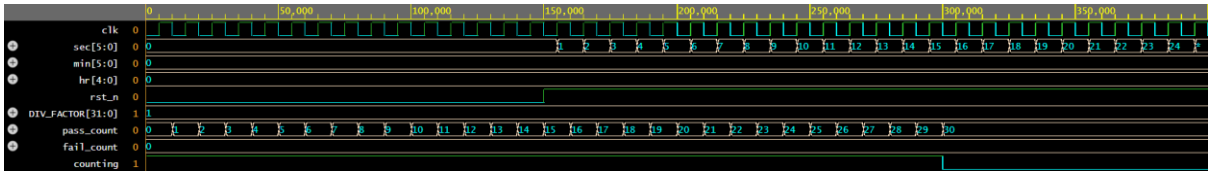

Figure 3: Waveform for `reset_test`

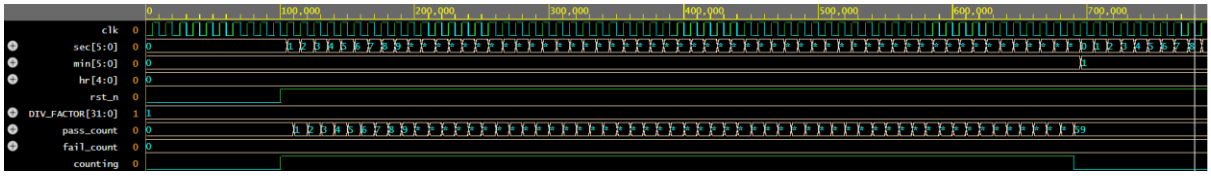

Figure 4: Waveform for `seconds_test`

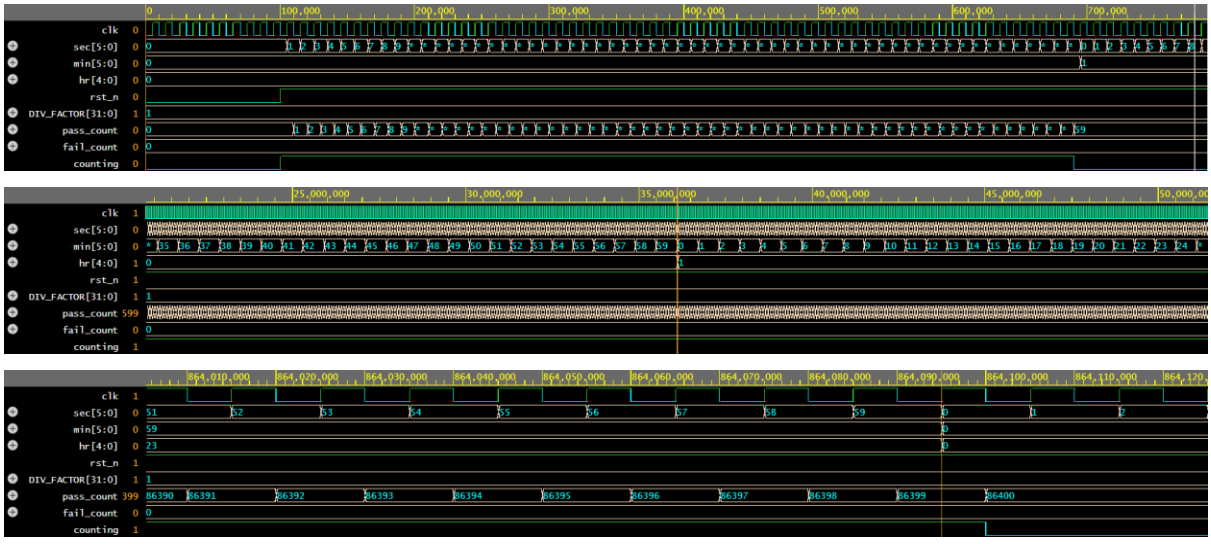

Figure 5: Waveforms for `rollover_test`

Full simulation logs for each test are available in Appendix B.

4 Future Work

Several enhancements and extensions can be considered for this project, including but not limited to:

- **Day and date counters:** the hour counter's rollover output is currently unused because the clock only counts up to 1 day. This could be connected to further counters, like a modulo-7 counter for a week, or a modulo-30 counter for a month, and so on.
- **Alarm functionality:** register-based comparison logic could be added to assert an interrupt when the clock's output values match a certain pre-programmed time.
- **Stopwatch and timer:** connected modes could be implemented to count up for a stopwatch indefinitely instead of rolling over, or a count down until zero for a timer.
- **12-hour format support:** the modulo-24 counter could be replaced with a modulo-12 counter for 12 hours, and then a modulo-2 counter for AM and PM.
- **BCD output:** the binary in the counter outputs could be converted to binary-coded decimal to allow for the direct drive of a display like a seven-segment display.

5 Conclusion

This project successfully designs, implements, and verifies a 24-hour digital clock in SystemVerilog with UVM. The RTL design follows a modular architecture, all instantiated in a top-level module. The UVM verification environment, including the targeted test sequences, confirms that the design is correct and efficient, as all 86,430 checked transitions across all tests pass with zero failures.

This project demonstrates a robust design and verification flow that can serve as the template for more complex digital systems.

6 References

IEEE. (2017). *IEEE Standard for SystemVerilog--Unified Hardware Design, Specification, and Verification Language.* IEEE.

IEEE. (2017). *IEEE Standard for Universal Verification Methodology Language Reference Manual.* IEEE.

SystemVerilog Project: Digital Clock (Ali Irfan). (2026, 07). Retrieved from EDA Playground: https://www.edaplayground.com/x/qgSM

APPENDICES

APPENDIX A: SOURCE CODE

```
`timescale 1ns / 1ps

//============================================================
// RTL Implementation of a Digital Clock
//============================================================

// Clock Divider: generates DIV_FACTOR enable pulse from fast clock
module clk_divider #(
    parameter DIV_FACTOR = 100_000_000  // clk = 10 ns -> x100,000,00 = 1 second
) (
    input  logic clk,
    input  logic rst_n,
    output logic en
);
    integer count;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            count <= 0;
        else if (count == DIV_FACTOR - 1)
            count <= 0;
        else
            count <= count + 1;
    end

    assign en = (count == DIV_FACTOR - 1);
endmodule



// Mod-60 Counter: counts 0 to 59 (used for seconds and minutes)
module mod60_counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       en,
    output logic [5:0] count,
    output logic       rollover
);
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            count <= 0;
        else if (en) begin
            if (count == 59)
                count <= 0;
            else
                count <= count + 1;
        end
    end

    assign rollover = en && (count == 59);
endmodule



// Mod-24 Counter: counts 0 to 23 (used for hours)
module mod24_counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       en,
    output logic [4:0] count,
    output logic       rollover
);
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            count <= 0;
        else if (en) begin
            if (count == 23)
                count <= 0;
            else
                count <= count + 1;
```

```
        end
    end

    assign rollover = en && (count == 23);
endmodule



// Digital Clock: top module
module digital_clock #(
    parameter DIV_FACTOR = 100_000_000  // clk = 10 ns -> x100,000,00 = 1 second
) (
    input  logic       clk,
    input  logic       rst_n,
    output logic [5:0] sec,
    output logic [5:0] min,
    output logic [4:0] hr
);
    logic en_sec;
    logic en_min;
    logic en_hr;

    clk_divider #(.DIV_FACTOR(DIV_FACTOR)) u_div (
        .clk    (clk),
        .rst_n  (rst_n),
        .en (en_sec)
    );

    mod60_counter u_sec (
        .clk      (clk),
        .rst_n    (rst_n),
        .en       (en_sec),
        .count    (sec),
        .rollover (en_min)
    );

    mod60_counter u_min (
        .clk      (clk),
        .rst_n    (rst_n),
        .en       (en_min),
        .count    (min),
        .rollover (en_hr)
    );

    mod24_counter u_hr (
        .clk      (clk),
        .rst_n    (rst_n),
        .en       (en_hr),
        .count    (hr),
        .rollover ()  // connected to nothing because no day counter
    );
endmodule
```

Listing A1: `rtl/digital_clock.sv`

```
`timescale 1ns / 1ps

//============================================================
// UVM Testbench Package
//============================================================

package clock_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"



    // Sequence Item

    class clk_seq_item extends uvm_sequence_item;
        `uvm_object_utils(clk_seq_item)

        rand bit reset_n;
        rand int duration;
        rand int div_factor;  // to control test speed
        bit counting;  // to avoid counting during setup/cleanup

        bit [5:0] sec;
        bit [5:0] min;
        bit [4:0] hr;

        constraint c_duration { duration > 0; duration < 100000; }

        function new(string name = "clk_seq_item");
            super.new(name);
            if (!uvm_config_db#(int)::get(null, "", "div_factor", div_factor))
                div_factor = 1;
            counting = 1;
            sec = 0;
            min = 0;
            hr = 0;
        endfunction
    endclass



    // Sequencer

    class clk_sequencer extends uvm_sequencer #(clk_seq_item);
        `uvm_component_utils(clk_sequencer)

        function new(string name = "clk_sequencer", uvm_component parent = null);
            super.new(name, parent);
        endfunction
    endclass



    // Driver

    class clk_driver extends uvm_driver #(clk_seq_item);
        `uvm_component_utils(clk_driver)

        virtual digital_clock_if vif;
        clk_seq_item req;

        function new(string name = "clk_driver", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            if (!uvm_config_db#(virtual digital_clock_if)::get(this, "", "vif", vif))
                `uvm_fatal("DRV", "No virtual interface found")
        endfunction

        task run_phase(uvm_phase phase);
            forever begin
                seq_item_port.get_next_item(req);
                vif.rst_n = req.reset_n;
```

```
            vif.counting = req.counting;
            repeat (req.duration) @(posedge vif.clk);
            @(negedge vif.clk);  // wait one extra negedge to avoid miscounting last test
            vif.counting = 0;
            seq_item_port.item_done();
        end
    endtask
endclass


// Monitor

class clk_monitor extends uvm_monitor;
    `uvm_component_utils(clk_monitor)

    virtual digital_clock_if vif;
    uvm_analysis_port #(clk_seq_item) mon_ap;

    function new(string name = "clk_monitor", uvm_component parent = null);
        super.new(name, parent);
        mon_ap = new("mon_ap", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual digital_clock_if)::get(this, "", "vif", vif))
            `uvm_fatal("MON", "No virtual interface found")
    endfunction

    task run_phase(uvm_phase phase);
        clk_seq_item item;
        forever begin
            @(negedge vif.clk);
            item = clk_seq_item::type_id::create("item");
            item.sec = vif.sec;
            item.min = vif.min;
            item.hr  = vif.hr;
            mon_ap.write(item);
        end
    endtask
endclass


// Agent

class clk_agent extends uvm_agent;
    `uvm_component_utils(clk_agent)

    clk_sequencer sqr;
    clk_driver drv;
    clk_monitor mon;

    function new(string name = "clk_agent", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sqr = clk_sequencer::type_id::create("sqr", this);
        drv = clk_driver::type_id::create("drv", this);
        mon = clk_monitor::type_id::create("mon", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        drv.seq_item_port.connect(sqr.seq_item_export);
    endfunction
endclass


// Scoreboard
```

```
    class clk_scoreboard extends uvm_scoreboard;
        `uvm_component_utils(clk_scoreboard)

        uvm_analysis_imp #(clk_seq_item, clk_scoreboard) mon_export;
        virtual digital_clock_if vif;

        int div_factor;
        int pass_count, fail_count;

        int clk_div_count;
        logic [5:0] model_sec;
        logic [5:0] model_min;
        logic [4:0] model_hr;

        function new(string name = "clk_scoreboard", uvm_component parent = null);
            super.new(name, parent);
            mon_export = new("mon_export", this);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            if (!uvm_config_db#(virtual digital_clock_if)::get(this, "", "vif", vif))
                `uvm_fatal("SCB", "No virtual interface")
            if (!uvm_config_db#(int)::get(this, "", "div_factor", div_factor))
                div_factor = 1;
            pass_count = 0;
            fail_count = 0;
        endfunction

        task run_phase(uvm_phase phase);
            forever begin
                @(posedge vif.clk);

                if (!vif.rst_n) begin
                    clk_div_count = 0;
                    model_sec = 0;
                    model_min = 0;
                    model_hr = 0;
                end else begin
                    logic en_sec = (clk_div_count == div_factor - 1);
                    logic en_min = en_sec && (model_sec == 59);
                    logic en_hr  = en_min  && (model_min == 59);

                    if (en_sec) begin
                        model_sec = (model_sec == 59) ? 0 : model_sec + 1;
                    end
                    if (en_min) begin
                        model_min = (model_min == 59) ? 0 : model_min + 1;
                    end
                    if (en_hr) begin
                        model_hr = (model_hr == 23) ? 0 : model_hr + 1;
                    end

                    if (clk_div_count == div_factor - 1)
                        clk_div_count = 0;
                    else
                        clk_div_count = clk_div_count + 1;
                end
            end
        endtask

        function void write(clk_seq_item item);
            bit sec_err, min_err, hr_err;

            if (!vif.counting) return;

            sec_err = (item.sec !== model_sec);
            min_err = (item.min !== model_min);
            hr_err  = (item.hr  !== model_hr);

            if (sec_err || min_err || hr_err) begin
                if (sec_err)
                    `uvm_error("SCB", $sformatf("SEC mismatch: DUT=%0d, REF=%0d", item.sec,
model_sec))
```

```
                if (min_err)
                    `uvm_error("SCB", $sformatf("MIN mismatch: DUT=%0d, REF=%0d", item.min,
model_min))
                if (hr_err)
                    `uvm_error("SCB", $sformatf("HR mismatch: DUT=%0d, REF=%0d", item.hr, model_hr))
                fail_count++;
            end else begin
                `uvm_info("SCB", $sformatf("MATCH: %0d:%0d:%0d", item.hr, item.min, item.sec),
UVM_HIGH)
                pass_count++;
            end
            vif.pass_count = pass_count;
            vif.fail_count = fail_count;
        endfunction

        function void report_phase(uvm_phase phase);
            `uvm_info("SCB", $sformatf("Scoreboard summary: %0d passed, %0d failed", pass_count,
fail_count), UVM_LOW)
        endfunction
    endclass



    // Environment

    class clk_env extends uvm_env;
        `uvm_component_utils(clk_env)

        clk_agent agt;
        clk_scoreboard scb;

        function new(string name = "clk_env", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            agt = clk_agent::type_id::create("agt", this);
            scb = clk_scoreboard::type_id::create("scb", this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            agt.mon.mon_ap.connect(scb.mon_export);
        endfunction
    endclass



    // Sequences

    class reset_seq extends uvm_sequence #(clk_seq_item);  // test active-low reset works
        `uvm_object_utils(reset_seq)

        function new(string name = "reset_seq");
            super.new(name);
        endfunction

        task body();
            clk_seq_item item;

            // 15 clock cycles with reset_n = 0 - clock should not tick
            item = clk_seq_item::type_id::create("item");
            start_item(item);
            item.reset_n = 0;
            item.duration = 15;
            finish_item(item);

            // 15 clock cycles with reset_n = 1 - clock should tick every DIV_FACTOR
            item = clk_seq_item::type_id::create("item");
            start_item(item);
            item.reset_n = 1;
            item.duration = 15;
            finish_item(item);
        endtask
```

```
endclass

class seconds_seq extends uvm_sequence #(clk_seq_item);  // test seconds update every DIV_FACTOR
    `uvm_object_utils(seconds_seq)

    int div_factor;

    function new(string name = "seconds_seq");
        super.new(name);
        if (!uvm_config_db#(int)::get(null, "", "div_factor", div_factor))
            div_factor = 1;
    endfunction

    task body();
        clk_seq_item item;

        // Set reset_n = 0 and allow to settle
        item = clk_seq_item::type_id::create("item");
        start_item(item);
        item.reset_n = 0;
        item.duration = 10;
        item.counting = 0;
        finish_item(item);

        // Let clock count up to DIV_FACTOR * 59
        item = clk_seq_item::type_id::create("item");
        start_item(item);
        item.reset_n = 1;
        item.duration = div_factor * 59;
        finish_item(item);
    endtask
endclass

class rollover_seq extends uvm_sequence #(clk_seq_item);  // test every combination
    `uvm_object_utils(rollover_seq)
    int div_factor;

    function new(string name = "rollover_seq");
        super.new(name);
        if (!uvm_config_db#(int)::get(null, "", "div_factor", div_factor))
            div_factor = 1;
    endfunction

    task body();
        clk_seq_item item;

        // Set reset_n = 0 and allow to settle
        item = clk_seq_item::type_id::create("item");
        start_item(item);
        item.reset_n = 0;
        item.duration = 10;
        item.counting = 0;
        finish_item(item);

        // Let clock count up one full day
        item = clk_seq_item::type_id::create("item");
        start_item(item);
        item.reset_n = 1;
        item.duration = div_factor * 24 * 60 * 60;
        finish_item(item);
    endtask
endclass


// Tests

class reset_test extends uvm_test;
    `uvm_component_utils(reset_test)

    clk_env env;
    reset_seq seq;

    function new(string name = "reset_test", uvm_component parent = null);
```

```
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = clk_env::type_id::create("env", this);
            seq = reset_seq::type_id::create("seq", this);
        endfunction

        task run_phase(uvm_phase phase);
            phase.raise_objection(this);
            seq.start(env.agt.sqr);
            #100;
            phase.drop_objection(this);
        endtask
    endclass

    class seconds_test extends uvm_test;
        `uvm_component_utils(seconds_test)

        clk_env env;
        seconds_seq seq;

        function new(string name = "seconds_test", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = clk_env::type_id::create("env", this);
            seq = seconds_seq::type_id::create("seq", this);
        endfunction

        task run_phase(uvm_phase phase);
            phase.raise_objection(this);
            seq.start(env.agt.sqr);
            #100;
            phase.drop_objection(this);
        endtask
    endclass

    class rollover_test extends uvm_test;
        `uvm_component_utils(rollover_test)

        clk_env env;
        rollover_seq seq;

        function new(string name = "rollover_test", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            env = clk_env::type_id::create("env", this);
            seq = rollover_seq::type_id::create("seq", this);
        endfunction

        task run_phase(uvm_phase phase);
            phase.raise_objection(this);
            seq.start(env.agt.sqr);
            #100;
            phase.drop_objection(this);
        endtask
    endclass

endpackage
```

Listing A2: `tb/clock_pkg.sv`

```
`timescale 1ns / 1ps

//============================================================
// Testbench Interface
//============================================================

interface digital_clock_if (input logic clk);
    logic rst_n;
    logic [5:0] sec;
    logic [5:0] min;
    logic [4:0] hr;
    int  pass_count, fail_count;
    logic counting;
endinterface
```

Listing A3: `tb/digital_clock_if.sv`

```
`timescale 1ns / 1ps

//============================================================
// Top Module
//============================================================

import uvm_pkg::*;
`include "uvm_macros.svh"
import clock_pkg::*;



module top;
    localparam int DIV_FACTOR = 1;

    logic clk;

    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;    // 10ns clock period (100MHz)
    end

    digital_clock_if vif (.*);

    digital_clock #(.DIV_FACTOR(DIV_FACTOR)) u_dut (
        .clk   (clk),
        .rst_n (vif.rst_n),
        .sec   (vif.sec),
        .min   (vif.min),
        .hr    (vif.hr)
    );

    initial begin
        uvm_config_db#(virtual digital_clock_if)::set(null, "*.agt.drv", "vif", vif);
        uvm_config_db#(virtual digital_clock_if)::set(null, "*.agt.mon", "vif", vif);
        uvm_config_db#(virtual digital_clock_if)::set(null, "*.scb", "vif", vif);
        uvm_config_db#(int)::set(null, "*", "div_factor", DIV_FACTOR);
    end

    initial begin
        run_test();
    end

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, top);
    end

endmodule
```

Listing A4: `tb/top.sv`

APPENDIX B: TEST LOGS

```
[2026-07-07 01:28:12 UTC] vlib work && vlog '-timescale' '1ns/1ns' '-sv2k9' +incdir+$RIVIERA_HOME/vlib/uvm-1.2/src -l uvm_1_2 -err
VCP2947 W9 -err VCP2974 W9 -err VCP3003 W9 -err VCP5417 W9 -err VCP6120 W9 -err VCP7862 W9 -err VCP2129 W9 design.sv testbench.sv  &&
vsim -c -do "vsim +access+r +UVM_TESTNAME=reset_test; run -all; exit"
VSIMSA: Configuration file changed: `/home/runner/library.cfg'
ALIB: Library "work" attached.
work = ./work/work.lib
MESSAGE_SP VCP2124 "Package uvm_pkg found in library uvm_1_2."
MESSAGE "Unit top modules: top."
SUCCESS "Compile success 0 Errors 0 Warnings  Analysis time: 6[s]."
done
# Aldec, Inc. Riviera-PRO version 2025.04.139.9738 built for Linux64 on May 30, 2025.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2025 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Warning: ELBREAD_0049 The "uvm_pkg" design unit does not have a time unit/precision defined but other design units do.
# ELBREAD: Elaboration time 0.8 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsystf.so'
# ELAB2: Create instances ...
# KERNEL: Info: Loading library:  /usr/share/Riviera-PRO/bin/uvm_1_2_dpi
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... done : 0.1 [s]
# SLP: Generation phase ...
# SLP: Generation phase ... done : 0.1 [s]
# SLP: Finished : 0.1 [s]
# SLP: 0 primitives and 10 (62.50%) other processes in SLP
# SLP: 43 (0.14%) signals in SLP and 9 (0.03%) interface signals
# ELAB2: Elaboration final pass complete - time: 2.6 [s].
# KERNEL: SLP loading done - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: SLP simulation initialization done - time: 0.0 [s].
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 29457 kB (elbread=2126 elab2=22483 kernel=4847 sdf=0)
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_root.svh(392) @ 0: reporter [UVM/RELNOTES]
# KERNEL: ----------------------------------------------------------------
# KERNEL: UVM-1.2
# KERNEL: (C) 2007-2014 Mentor Graphics Corporation
# KERNEL: (C) 2007-2014 Cadence Design Systems, Inc.
# KERNEL: (C) 2006-2014 Synopsys, Inc.
# KERNEL: (C) 2011-2013 Cypress Semiconductor Corp.
# KERNEL: (C) 2013-2014 NVIDIA Corporation
# KERNEL: ----------------------------------------------------------------
# KERNEL:
# KERNEL:   ***********       IMPORTANT RELEASE NOTES         ************
# KERNEL:
# KERNEL:   You are using a version of the UVM library that has been compiled
# KERNEL:   with `UVM_NO_DEPRECATED undefined.
# KERNEL:   See http://www.eda.org/svdb/view.php?id=3313 for more details.
# KERNEL:
# KERNEL:   You are using a version of the UVM library that has been compiled
# KERNEL:   with `UVM_OBJECT_DO_NOT_NEED_CONSTRUCTOR undefined.
# KERNEL:   See http://www.eda.org/svdb/view.php?id=3770 for more details.
# KERNEL:
# KERNEL:       (Specify +UVM_NO_RELNOTES to turn off this notice)
# KERNEL:
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: UVM_INFO @ 0: reporter [RNTST] Running test reset_test...
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_objection.svh(1271) @ 400000: reporter [TEST_DONE] 'run' phase is ready to proceed to the
'extract' phase
# KERNEL: UVM_INFO /home/runner/testbench.sv(247) @ 400000: uvm_test_top.env.scb [SCB] Scoreboard summary: 30 passed, 0 failed
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_report_server.svh(869) @ 400000: reporter [UVM/REPORT/SERVER]
# KERNEL: --- UVM Report Summary ---
# KERNEL:
# KERNEL: ** Report counts by severity
# KERNEL: UVM_INFO :    4
# KERNEL: UVM_WARNING :    0
# KERNEL: UVM_ERROR :    0
# KERNEL: UVM_FATAL :    0
# KERNEL: ** Report counts by id
# KERNEL: [RNTST]     1
# KERNEL: [SCB]     1
# KERNEL: [TEST_DONE]     1
# KERNEL: [UVM/RELNOTES]     1
# KERNEL:
# RUNTIME: Info: RUNTIME_0068 uvm_root.svh (521): $finish called.
# KERNEL: Time: 400 ns,  Iteration: 57,  Instance: /top,  Process: @INITIAL#481_2@.
# KERNEL: stopped at time: 400 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2026-07-07 01:28:26 UTC] Opening EPWave...
Done
```

Listing B1: Simulation logs for `reset_test`

```
[2026-07-07 01:26:32 UTC] vlib work && vlog '-timescale' '1ns/1ns' '-sv2k9' +incdir+$RIVIERA_HOME/vlib/uvm-1.2/src -l uvm_1_2 -err
VCP2947 W9 -err VCP2974 W9 -err VCP3003 W9 -err VCP5417 W9 -err VCP6120 W9 -err VCP7862 W9 -err VCP2129 W9 design.sv testbench.sv  &&
vsim -c -do "vsim +access+r +UVM_TESTNAME=seconds_test; run -all; exit"
VSIMSA: Configuration file changed: `/home/runner/library.cfg'
ALIB: Library "work" attached.
work = ./work/work.lib
MESSAGE_SP VCP2124 "Package uvm_pkg found in library uvm_1_2."
MESSAGE "Unit top modules: top."
SUCCESS "Compile success 0 Errors 0 Warnings  Analysis time: 7[s]."
done
# Aldec, Inc. Riviera-PRO version 2025.04.139.9738 built for Linux64 on May 30, 2025.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2025 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Warning: ELBREAD_0049 The "uvm_pkg" design unit does not have a time unit/precision defined but other design units do.
# ELBREAD: Elaboration time 0.8 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsystf.so'
# ELAB2: Create instances ...
# KERNEL: Info: Loading library:  /usr/share/Riviera-PRO/bin/uvm_1_2_dpi
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... done : 0.0 [s]
# SLP: Generation phase ...
# SLP: Generation phase ... done : 0.1 [s]
# SLP: Finished : 0.1 [s]
# SLP: 0 primitives and 10 (62.50%) other processes in SLP
# SLP: 43 (0.14%) signals in SLP and 9 (0.03%) interface signals
# ELAB2: Elaboration final pass complete - time: 2.5 [s].
# KERNEL: SLP loading done - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: SLP simulation initialization done - time: 0.0 [s].
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 29457 kB (elbread=2126 elab2=22483 kernel=4847 sdf=0)
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_root.svh(392) @ 0: reporter [UVM/RELNOTES]
# KERNEL: ----------------------------------------------------------------
# KERNEL: UVM-1.2
# KERNEL: (C) 2007-2014 Mentor Graphics Corporation
# KERNEL: (C) 2007-2014 Cadence Design Systems, Inc.
# KERNEL: (C) 2006-2014 Synopsys, Inc.
# KERNEL: (C) 2011-2013 Cypress Semiconductor Corp.
# KERNEL: (C) 2013-2014 NVIDIA Corporation
# KERNEL: ----------------------------------------------------------------
# KERNEL:
# KERNEL:   ***********       IMPORTANT RELEASE NOTES         ************
# KERNEL:
# KERNEL:   You are using a version of the UVM library that has been compiled
# KERNEL:   with `UVM_NO_DEPRECATED undefined.
# KERNEL:   See http://www.eda.org/svdb/view.php?id=3313 for more details.
# KERNEL:
# KERNEL:   You are using a version of the UVM library that has been compiled
# KERNEL:   with `UVM_OBJECT_DO_NOT_NEED_CONSTRUCTOR undefined.
# KERNEL:   See http://www.eda.org/svdb/view.php?id=3770 for more details.
# KERNEL:
# KERNEL:       (Specify +UVM_NO_RELNOTES to turn off this notice)
# KERNEL:
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: UVM_INFO @ 0: reporter [RNTST] Running test seconds_test...
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_objection.svh(1271) @ 790000: reporter [TEST_DONE] 'run' phase is ready to proceed to the
'extract' phase
# KERNEL: UVM_INFO /home/runner/testbench.sv(247) @ 790000: uvm_test_top.env.scb [SCB] Scoreboard summary: 59 passed, 0 failed
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_report_server.svh(869) @ 790000: reporter [UVM/REPORT/SERVER]
# KERNEL: --- UVM Report Summary ---
# KERNEL:
# KERNEL: ** Report counts by severity
# KERNEL: UVM_INFO :    4
# KERNEL: UVM_WARNING :    0
# KERNEL: UVM_ERROR :    0
# KERNEL: UVM_FATAL :    0
# KERNEL: ** Report counts by id
# KERNEL: [RNTST]     1
# KERNEL: [SCB]     1
# KERNEL: [TEST_DONE]     1
# KERNEL: [UVM/RELNOTES]     1
# KERNEL:
# RUNTIME: Info: RUNTIME_0068 uvm_root.svh (521): $finish called.
# KERNEL: Time: 790 ns,  Iteration: 57,  Instance: /top,  Process: @INITIAL#481_2@.
# KERNEL: stopped at time: 790 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2026-07-07 01:26:46 UTC] Opening EPWave...
Done
```

Listing B2: Simulation logs for `seconds_test`

```
[2026-07-07 10:33:50 UTC] vlib work && vlog '-timescale' '1ns/1ns' '-sv2k9' +incdir+$RIVIERA_HOME/vlib/uvm-1.2/src -l uvm_1_2 -err
VCP2947 W9 -err VCP2974 W9 -err VCP3003 W9 -err VCP5417 W9 -err VCP6120 W9 -err VCP7862 W9 -err VCP2129 W9 design.sv testbench.sv  &&
vsim -c -do "vsim +access+r +UVM_TESTNAME=rollover_test; run -all; exit"
VSIMSA: Configuration file changed: `/home/runner/library.cfg'
ALIB: Library "work" attached.
work = ./work/work.lib
MESSAGE_SP VCP2124 "Package uvm_pkg found in library uvm_1_2."
MESSAGE "Unit top modules: top."
SUCCESS "Compile success 0 Errors 0 Warnings  Analysis time: 6[s]."
done
# Aldec, Inc. Riviera-PRO version 2025.04.139.9738 built for Linux64 on May 30, 2025.
# HDL, SystemC, and Assertions simulator, debugger, and design environment.
# (c) 1999-2025 Aldec, Inc. All rights reserved.
# ELBREAD: Elaboration process.
# ELBREAD: Warning: ELBREAD_0049 The "uvm_pkg" design unit does not have a time unit/precision defined but other design units do.
# ELBREAD: Elaboration time 0.8 [s].
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# KERNEL: PLI/VHPI kernel's engine initialization done.
# PLI: Loading library '/usr/share/Riviera-PRO/bin/libsystf.so'
# ELAB2: Create instances ...
# KERNEL: Info: Loading library:  /usr/share/Riviera-PRO/bin/uvm_1_2_dpi
# KERNEL: Time resolution set to 1ps.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... done : 0.0 [s]
# SLP: Generation phase ...
# SLP: Generation phase ... done : 0.1 [s]
# SLP: Finished : 0.1 [s]
# SLP: 0 primitives and 10 (62.50%) other processes in SLP
# SLP: 43 (0.14%) signals in SLP and 9 (0.03%) interface signals
# ELAB2: Elaboration final pass complete - time: 2.7 [s].
# KERNEL: SLP loading done - time: 0.0 [s].
# KERNEL: Warning: You are using the Riviera-PRO EDU Edition. The performance of simulation is reduced.
# KERNEL: Warning: Contact Aldec for available upgrade options - sales@aldec.com.
# KERNEL: SLP simulation initialization done - time: 0.0 [s].
# KERNEL: Kernel process initialization done.
# Allocation: Simulator allocated 29457 kB (elbread=2126 elab2=22482 kernel=4847 sdf=0)
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_root.svh(392) @ 0: reporter [UVM/RELNOTES]
# KERNEL: ----------------------------------------------------------------
# KERNEL: UVM-1.2
# KERNEL: (C) 2007-2014 Mentor Graphics Corporation
# KERNEL: (C) 2007-2014 Cadence Design Systems, Inc.
# KERNEL: (C) 2006-2014 Synopsys, Inc.
# KERNEL: (C) 2011-2013 Cypress Semiconductor Corp.
# KERNEL: (C) 2013-2014 NVIDIA Corporation
# KERNEL: ----------------------------------------------------------------
# KERNEL:
# KERNEL:   ***********       IMPORTANT RELEASE NOTES         ************
# KERNEL:
# KERNEL:   You are using a version of the UVM library that has been compiled
# KERNEL:   with `UVM_NO_DEPRECATED undefined.
# KERNEL:   See http://www.eda.org/svdb/view.php?id=3313 for more details.
# KERNEL:
# KERNEL:   You are using a version of the UVM library that has been compiled
# KERNEL:   with `UVM_OBJECT_DO_NOT_NEED_CONSTRUCTOR undefined.
# KERNEL:   See http://www.eda.org/svdb/view.php?id=3770 for more details.
# KERNEL:
# KERNEL:       (Specify +UVM_NO_RELNOTES to turn off this notice)
# KERNEL:
# KERNEL: ASDB file was created in location /home/runner/dataset.asdb
# KERNEL: UVM_INFO @ 0: reporter [RNTST] Running test rollover_test...
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_objection.svh(1271) @ 864200000: reporter [TEST_DONE] 'run' phase is ready to proceed to the
'extract' phase
# KERNEL: UVM_INFO /home/runner/testbench.sv(247) @ 864200000: uvm_test_top.env.scb [SCB] Scoreboard summary: 86400 passed, 0 failed
# KERNEL: UVM_INFO ./uvm-1.2/src/base/uvm_report_server.svh(869) @ 864200000: reporter [UVM/REPORT/SERVER]
# KERNEL: --- UVM Report Summary ---
# KERNEL:
# KERNEL: ** Report counts by severity
# KERNEL: UVM_INFO :    4
# KERNEL: UVM_WARNING :    0
# KERNEL: UVM_ERROR :    0
# KERNEL: UVM_FATAL :    0
# KERNEL: ** Report counts by id
# KERNEL: [RNTST]     1
# KERNEL: [SCB]     1
# KERNEL: [TEST_DONE]     1
# KERNEL: [UVM/RELNOTES]     1
# KERNEL:
# RUNTIME: Info: RUNTIME_0068 uvm_root.svh (521): $finish called.
# KERNEL: Time: 864200 ns,  Iteration: 57,  Instance: /top,  Process: @INITIAL#479_2@.
# KERNEL: stopped at time: 864200 ns
# VSIM: Simulation has finished. There are no more test vectors to simulate.
# VSIM: Simulation has finished.
Finding VCD file...
./dump.vcd
[2026-07-07 10:34:32 UTC] Opening EPWave...
Done
```

Listing B3: Simulation logs for `rollover_test`